SmartTask

Web Business Application

Final Project Report

Course Title & Code: Web Business Applications / BIS 351

Department: Management Information Systems

College of Business Administration

Imam Abdulrahman Bin Faisal University

Academic Year: 2024-2025

Group Members

Student Name	Student ID
Abdulrhman Albelale	2240001189
Mohammad Almejnah	2240000470
Bader Alotaibi	2240002142
Abdulmohsn Alkhaldi	2240000769

Table of Contents

Project Overview

SmartTask is a browser-based task management web application developed for the BIS 351 Web Business Applications final project. The application addresses a practical business and academic problem: many students and busy users need a simple way to record, prioritize, and follow up on tasks, but many existing tools require accounts, subscriptions, advanced setup, or learning time. SmartTask focuses on a smaller and clearer promise: open the website, add a task, select its priority and due date, connect it to a goal, and continue working without unnecessary steps.

The project was built as a functional front-end prototype using HTML5, CSS3, and JavaScript. Task data is stored in the browser through localStorage, so the application works without a database or server-side login. This technical choice supports the project value proposition: fast access, privacy, low cost, and ease of use. The report explains the business problem, target users, features, system architecture, user interface design, development workflow, testing results, and ethical considerations. It also includes visual evidence such as screenshots, tables, a Gantt chart, and an application structure diagram to meet the project requirements.

Overall, SmartTask demonstrates how a focused business web application can solve a real user need without becoming complicated. The project meets the main course learning outcomes by applying HTML, CSS, and scripting to develop a business-oriented web application and by discussing professional, ethical, and legal responsibilities related to IT usage.

2. Introduction and Project Background

In academic and professional life, people often need to handle many responsibilities at the same time. A university student may have assignments, quizzes, group projects, presentations, registration deadlines, and personal goals. A working person may have meetings, reports, client requests, and follow-up tasks. When these responsibilities are not written down and prioritized clearly, users may miss deadlines, forget important work, or waste time deciding what to do next. Time management literature explains that planning and perceived control over time are strongly connected with performance and lower stress levels (Britton and Tesser, 1991; Macan et al., 1990).

There are many digital productivity tools available in the market, such as Trello, Notion, Todoist, and Microsoft To Do. These tools are useful, but they are not always suitable for a user who wants a quick and lightweight solution. Some require account creation, cloud storage, subscriptions, or a complex workspace structure. In many cases, the tool itself becomes another task to manage. SmartTask was designed to avoid this problem by giving users only the essential features needed for everyday task planning.

The name SmartTask reflects the project objective. The application is not designed to replace full project management platforms. Instead, it is designed for fast personal planning, especially for students and individuals who need clarity more than complexity. The core idea is that task management should be simple enough to use daily and structured enough to improve decision making. By combining task title, description, priority, due date, time, goal link, and reason for importance, SmartTask encourages users to think not only about what they need to do, but also why the task matters.

The project is relevant to Web Business Applications because it solves a real business and user problem through a web interface. It demonstrates the role of front-end technologies in creating useful digital services and shows how business needs can guide technical decisions. The final prototype includes multiple pages, navigation, interactive forms, local data persistence, task editing, completion, deletion, and support information.

3. Business Problem and Target Market

3.1 Problem Statement

The main problem addressed by SmartTask is the lack of a simple, no-login, browser-based task planner for users who need to organize daily and weekly responsibilities without being overwhelmed by advanced project management features. Many users understand that they need a task system, but they do not always continue using existing applications because the setup process is too long, the interface is too crowded, or the useful features are locked behind subscriptions.

The problem can be stated as follows: students and busy individuals need an accessible and privacy-respecting way to add, prioritize, schedule, and review tasks from any browser without creating an account, installing software, or learning a complex productivity platform. This problem is important because poor task management can lead to missed deadlines, stress, lower academic performance, and inefficient daily routines (Claessens et al., 2007).

3.2 Target Users

Target Segment	Main Pain Point	How SmartTask Responds
University students	Many deadlines across different courses and activities	Simple task entry, priority badges, due date options, and goal links
Working professionals	Fast task inflow and need for quick organization	No login, quick browser access, edit/complete/delete actions
Freelancers and small-business users	Need a free lightweight tool without enterprise complexity	Zero-cost front-end solution with local storage
General users	Paper notes get lost and complex apps are hard to maintain	Clean interface with only essential planning features

3.3 Market Gap and Opportunity

The task management market is strong, but most popular products compete by adding more integrations, templates, automation, team dashboards, and paid plans. This creates a gap for users who want the opposite experience: fewer decisions, fewer screens, and faster usage. SmartTask focuses on simplicity and ease of use. It does not try to be the most powerful task platform. Instead, it tries to be the easiest task planner to open and use immediately.

From a business perspective, this position is valuable because it targets an underserved segment. A lightweight tool can attract users who are not ready for complex systems. It can also become a first step before future monetization options such as optional cloud sync, export features, or premium themes. However, the current academic prototype focuses on usefulness and proof of concept rather than revenue generation.

3.4 Data and Information Used for the Report

The report was built using both secondary and project-based information. Secondary information includes academic sources on time management, usability, accessibility, and web storage. These sources support the argument that structured planning improves control, reduces stress, and improves user performance. Project-based information includes the team's own application screens, feature decisions, file structure, and testing observations. Combining these two sources makes the report stronger because it connects theory with the actual SmartTask prototype.

The team also used informal usability observation while reviewing the prototype. The main question during review was not only whether the feature worked, but whether a normal student could understand it quickly. This led to practical decisions such as keeping the task form on one page, using clear priority buttons, highlighting delete in red, and showing completed tasks visually instead of hiding them immediately.

4. Proposed Solution and Core Features

SmartTask solves the identified problem through a multi-page web application that focuses on task creation and task management. The solution is based on five design ideas: simplicity, privacy, accessibility, speed, and motivation. Users should be able to understand the interface without instructions. Their data should stay on their device. The layout should be readable on different screen sizes. Actions should be immediate. Finally, users should be encouraged to connect tasks to their personal goals, not only deadlines.

4.1 Main Pages

- Home page: introduces the application and guides users to start planning.
- Tasks page: contains the main task creation form and the user task list.
- About page: explains the purpose of SmartTask and the team motivation.
- Support page: provides contact options for help and user support.

4.2 Task Creation Functionality

The task creation form is the core feature of the application. It allows the user to enter a task title, description, priority level, due date option, time, linked goal, and reason for importance. The priority selection uses High, Medium, and Low badges so that urgent work can be recognized quickly. Due date options include Today, Tomorrow, Daily, and Custom, allowing both short-term and repeated planning. The optional goal and reason fields are included to support reflective planning and motivation.

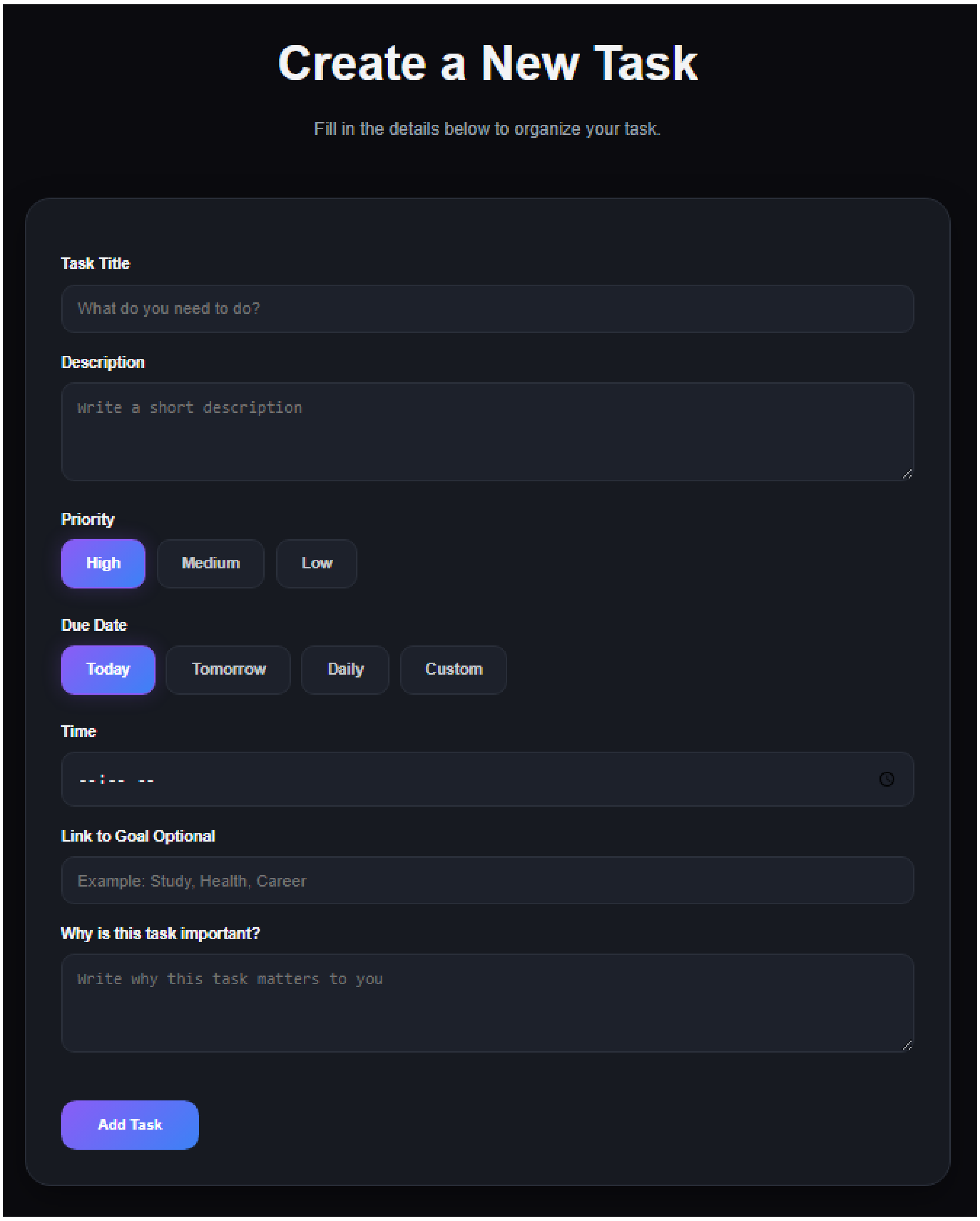

Figure 1: SmartTask task creation form

4.3 Task Management Functionality

After a task is added, it appears in the task list with its full information. The user can mark it as complete, edit it, or delete it. Completion changes the task status and gives immediate visual feedback. Editing reuses the form so the user can update details without creating a duplicate task. Deletion is highlighted clearly because removing a task is a sensitive action that should not happen accidentally. These actions cover the full CRUD cycle: Create, Read, Update, and Delete.

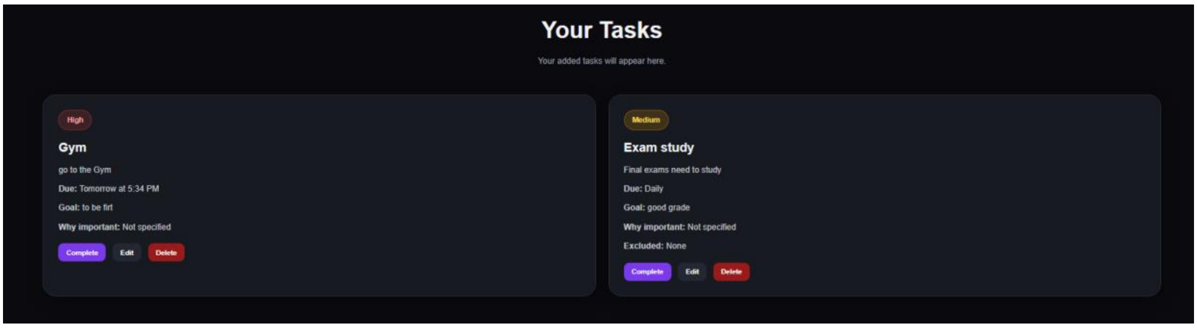

Figure 2: SmartTask user task list with priority cards

4.4 Feature Comparison

Feature	SmartTask	Trello	Notion	Todoist
No login needed for basic use	Yes	No	No	No
Local data storage	Yes	No	No	No
Priority levels	Yes	Yes	Yes	Yes
Goal linkage field	Yes	Limited	Yes	Limited
Zero setup time	High	Medium	Low	Medium
Best fit	Simple personal planning	Kanban projects	Knowledge/workspace management	Task lists and reminders

5. System Architecture and Technology Stack

SmartTask uses a client-side architecture. This means the application runs inside the browser and does not depend on a back-end server, user authentication, or an external database. This architecture is suitable for the project scope because the application is designed for personal task planning rather than enterprise collaboration. It also supports privacy because task data remains stored in the user browser.

5.1 Technology Stack

Technology	Use in SmartTask	Reason for Selection
HTML5	Builds the page structure, forms, buttons, navigation, and semantic content	It is the standard language for web page structure and supports the course learning outcome
CSS3	Controls layout, colors, spacing, cards, responsiveness, and visual hierarchy	It allows a polished interface without heavy frameworks
JavaScript	Handles task creation, editing, deletion, completion, dynamic fields, and localStorage	It provides interactivity and business logic on the client side
localStorage	Stores task data in the browser as JSON	It enables persistence without a database or login
GitHub	Stores source code and tracks development history	It supports teamwork, version control, and professional submission

5.2 Application Structure

File / Folder	Purpose
index.html	Landing page and navigation entry point
tasks.html	Main application page for task creation and task list
about.html	Information about the project purpose and team
support.html	Support channels and help information
css/style.css	Global styling and shared layout rules
css/tasks.css	Specific styling for task cards and task form
css/support.css	Specific styling for support page content
js/app.js	Task logic, event listeners, validation, CRUD functions, and localStorage operations
README.md	Project description, setup instructions, screenshots, and repository information

5.3 Data Model and Storage

Each task is represented as a JavaScript object. A typical object contains fields such as title, description, priority, due date, date, time, goal, reason, excluded days, and completion status. The task array is converted to JSON and saved under a localStorage key. When the page loads again, the JavaScript code reads the stored JSON, converts it back to objects, and renders the task cards. This design is simple, transparent, and appropriate for a lightweight prototype.

Simplified task object schema:

{ title, description, priority, due, date, time, goal, reason, excludedDays, completed }

5.4 Architecture Diagram

User Browser	SmartTask Front-End	Browser Storage
User enters task information and clicks Add Task	HTML form + CSS interface + JavaScript validation and CRUD logic	localStorage saves tasks as JSON and reloads them in the next session

6. UI/UX Design and Wireframes

The user interface was designed around clarity and quick usage. The main tasks page uses a card-based layout with a dark theme, strong contrast, rounded containers, and clear button states. Important information is grouped logically: task details first, then priority, then due date, then motivational fields. The user does not need to move across many screens to create a task.

6.1 Design Principles

- Clarity: labels are direct and fields are arranged in the order a user naturally thinks about a task.
- Visual hierarchy: titles, priority badges, action buttons, and due dates are visually separated.
- Responsiveness: the interface is intended to adjust for both desktop and mobile screens.
- Feedback: actions such as completion, edit, and delete give immediate response to the user.
- Accessibility: readable text sizes, labeled form fields, and contrast-conscious color choices improve usability.

6.2 Color and Typography Decisions

Element	Design Choice	Purpose
Primary color	Blue / violet gradient accents	Creates a modern technology feel and highlights main actions
Background	Dark neutral theme	Reduces visual distraction and helps cards stand out
High priority	Red badge	Signals urgent tasks
Medium priority	Amber badge	Signals moderate urgency
Low priority	Neutral/darker badge	Signals lower urgency
Typography	Clean sans-serif in the application; Times New Roman in the report	Supports screen readability in the app and academic formatting in the report

6.3 Wireframe Summary

The initial wireframe followed a simple vertical flow: header navigation, page title, task form, and task list. This design was chosen because it matches the user journey. The user first decides what to add, then reviews what has already been created. Keeping both the form and the list on the same page reduces navigation friction and makes the application more practical for daily use.

7. GitHub Repository and Development Process

The project should be submitted through a GitHub repository containing the full application source code and a README file. GitHub is important because it provides evidence of development progress, allows group members to collaborate, and makes it easier for the instructor to review the project files. Regular commits also show that the work was built progressively rather than assembled at the last moment.

7.1 Repository Requirements

- Repository name: SmartTask or SmartTask-BIS351.
- README.md: project overview, features, technology stack, instructions to run the project, screenshots, and team members.
- Source code folders: html pages, css folder, js folder, and assets folder if needed.
- Commit history: meaningful commit messages such as "Create tasks page", "Add localStorage saving", and "Improve responsive design".
- Live demo link: optional, but recommended if hosted on GitHub Pages.

GitHub repository link: [https://github.com/alkhaldi80/Smart-Task-Planner]

7.2 Gantt Chart and Timeline

Activity	W1-2	W3-4	W5-6	W7-8	W9-10	W11-13
Idea & problem	X					
UI/UX prototype		X	X			
GitHub & README		X				
HTML pages			X	X		
CSS & responsive			X	X	X	
JavaScript logic				X	X	
Testing					X	X
Report					X	X

7.3 Teamwork and Responsibility

A successful group project requires clear responsibilities. The recommended workflow is to assign one member to coordinate the report, one member to manage HTML structure, one member to manage styling and UI improvements, and one member to manage JavaScript logic and testing. All members should still understand the full project because the course policy allows the instructor to ask any student to explain any part of the work individually.

8. Testing and Validation

Testing was planned around the main user journey: open the application, create a task, review the task list, edit a task, complete a task, delete a task, and return later to verify persistence. Because SmartTask is a front-end prototype, manual functional testing is appropriate for the current stage. The testing process focused on accuracy, usability, responsiveness, and data persistence.

8.1 Functional Testing

Test Case	Expected Result	Status
Add a task with all fields completed	Task appears with correct title, priority, date, time, goal, and reason	Pass
Add a task with only required information	Task is created and optional fields show default or empty values	Pass
Select High, Medium, and Low priority	Task card displays the correct badge style	Pass
Mark a task as complete	Task status changes and the card shows completion feedback	Pass
Edit an existing task	Form is populated and saved changes update the task card	Pass
Delete a task	Task is removed from the interface and localStorage	Pass
Close and reopen browser	Saved tasks reload correctly from localStorage	Pass
Test on smaller screen size	Form and cards remain readable and usable	Pass

8.2 Validation Against Project Requirements

Requirement	Evidence in SmartTask
Main pages and navigation	Home, Tasks, About, and Support pages
Core business functionality	Task planning with CRUD operations and local storage
Semantic HTML, CSS, scripting	HTML pages, CSS files, and JavaScript app logic
GitHub repository	Repository should include source code and README
Report with visuals	This report includes tables, screenshots, diagrams, and Gantt chart
Ethical and legal awareness	Privacy, accessibility, intellectual property, and AI disclosure sections

8.3 Limitations Found During Testing

The current prototype has some limitations. localStorage is convenient, but it does not synchronize data across devices. It also depends on the browser; if the user clears browser data, tasks may be removed. The application currently does not send automatic reminders or notifications. These limitations are acceptable for the academic prototype, but they guide the future enhancement plan.

8.4 Project Limitations

The current version of SmartTask is a simple academic prototype. Some features such as cloud synchronization and automatic reminders are not included yet, but they can be added in future versions.

9. Ethical and Legal Considerations

Ethical and legal responsibility is important in any web business application. SmartTask is a small prototype, but it still handles user-entered task information. Even simple task details may include personal goals, deadlines, or private notes. Therefore, the project was designed to limit data collection and avoid unnecessary risk.

9.1 Data Privacy

SmartTask does not require registration and does not send user tasks to an external server. All data is stored locally in the browser. This design follows the principle of data minimization because the application collects only what is necessary for its function. It also reduces privacy risk because there is no central database containing user tasks. However, users should understand that local browser storage is not the same as encrypted cloud storage, so they should avoid entering highly sensitive information.

9.2 Accessibility and Inclusion

The interface uses clear labels and strong visual contrast to support usability. Buttons are named according to their function, and fields are grouped in a predictable order. Future versions should continue improving accessibility by testing keyboard navigation, screen reader behavior, and contrast ratios against WCAG guidance (W3C, 2018).

9.3 Intellectual Property

The application source code and report content should be original work of the group. Any external assets, icons, images, frameworks, or copied code must be credited properly. If the team uses open-source snippets or libraries, the license should be checked before submission. This protects the team from plagiarism and intellectual property issues.

9.4 Generative AI Usage Disclosure

Generative AI tools may be used only within the allowed course limit and only for appropriate support activities such as brainstorming, proofreading, improving language, and understanding concepts. The team remains responsible for the final submission and should be able to explain the code, design choices, and report content orally. The final work should reflect the team's own understanding and implementation.

10. Conclusion and Future Enhancements

SmartTask demonstrates a practical solution to a common task management problem. The project shows that a useful business web application does not always require a complex back-end system or a large technology stack. By focusing on clear user needs, the team created a functional prototype that allows users to add, prioritize, schedule, edit, complete, and delete tasks in a straightforward interface.

The strongest part of SmartTask is its focus. It does not attempt to copy every feature of large productivity platforms. Instead, it provides the essential features that everyday users need most. The use of localStorage supports privacy and removes the need for account creation. The user interface supports quick planning and visual decision making through priority badges and task cards.

10.1 Achieved Objectives

- Developed a functional web application using HTML, CSS, and JavaScript.
- Implemented task creation, reading, editing, completion, deletion, and browser persistence.
- Designed a clear and responsive task management interface.
- Prepared professional documentation with analysis, visuals, testing, and ethical discussion.
- Aligned the project with the BIS 351 focus on web business applications and IT responsibility.

10.2 Future Enhancements

1. Task categories and tags: allow users to organize tasks by course, project, work area, or personal goal.
2. Browser notifications: remind users before a due time using the Web Notifications API.
3. Export options: allow users to export tasks as PDF or CSV for backup and sharing.
4. Dark and light mode: give users a choice based on environment and preference.
5. Drag-and-drop ordering: allow users to manually rearrange tasks by importance.
6. Optional cloud synchronization: provide secure account-based sync for users who need multi-device access.
7. Analytics dashboard: show completed tasks, pending tasks, and productivity trends over time.

11. References

Allen, D. (2001) Getting Things Done: The Art of Stress-Free Productivity. New York: Viking.

Britton, B.K. and Tesser, A. (1991) "Effects of time-management practices on college grades", Journal of Educational Psychology, 83(3), pp. 405-410.

Claessens, B.J.C., van Eerde, W., Rutte, C.G. and Roe, R.A. (2007) "A review of the time management literature", Personnel Review, 36(2), pp. 255-276.

Macan, T.H., Shahani, C., Dipboye, R.L. and Phillips, A.P. (1990) "College students' time management: Correlations with academic performance and stress", Journal of Educational Psychology, 82(4), pp. 760-768.

Mozilla Developer Network (2024) Window: localStorage property. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage (Accessed: 10 May 2025).

Nielsen, J. (1994) Usability Engineering. Boston: Academic Press.

W3C (2018) Web Content Accessibility Guidelines (WCAG) 2.1. Available at: https://www.w3.org/TR/WCAG21/ (Accessed: 10 May 2025).